



Безперервна інтеграція GitLab (GitLab CI/CD)

БЕЗПЕРЕРВНА ІНТЕГРАЦІЯ

- Безперервна інтеграція — це практика розробки програмного забезпечення, у якій ви **створюєте та тестуєте** програмне забезпечення кожного разу, коли розробник надсилає код до програми, і це відбувається кілька разів на день.

Безперервна інтеграція: TEST - BUILD

БЕЗПЕРЕРВНА ДОСТАВКА

- Continuous Delivery — це підхід до розробки програмного забезпечення, у якому безперервна інтеграція, автоматичне тестування та можливості автоматизованого розгортання дозволяють швидко, надійно та неодноразово розробляти та розгортати програмне забезпечення з мінімальним людським втручанням. Проте розгортання у виробництві визначається стратегічно та запускається вручну.

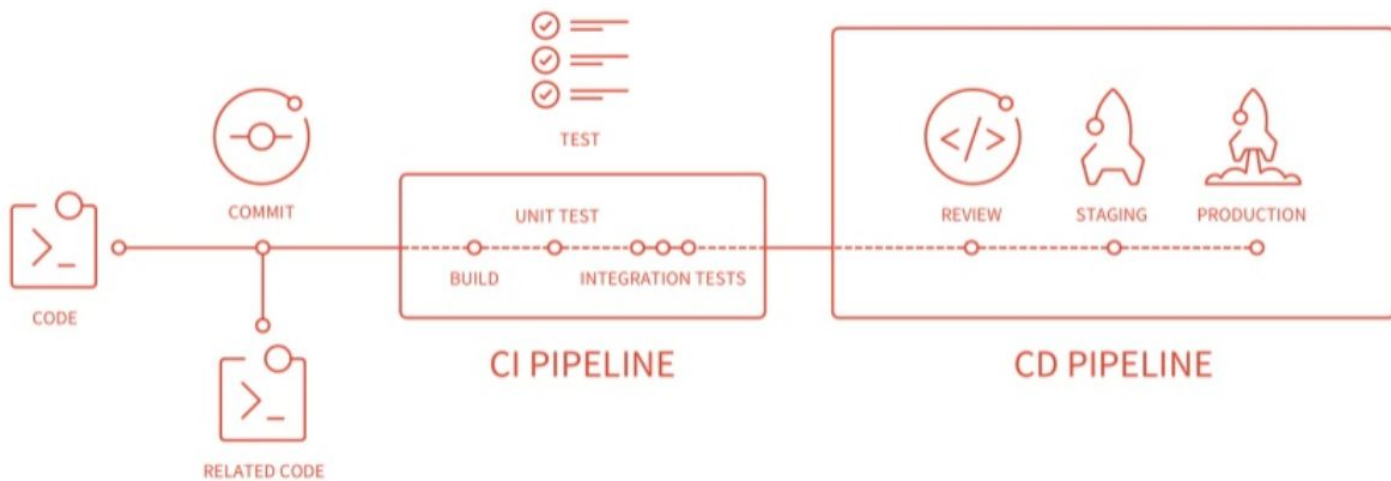
**Безперервна доставка: ТЕСТУВАННЯ –
СТВОРЕННЯ – РОЗГОРТАННЯ ВРУЧНУ**

БЕЗПЕРЕРВНЕ РОЗГОРТАННЯ

- Безперервне розгортання — це практика розробки програмного забезпечення, у якій кожна зміна коду проходить через увесь конвеєр і автоматично запускається у виробництво, що призводить до багатьох виробничих розгортань щодня. Він робить усе, що й Continuous Delivery, але процес повністю автоматизований, без жодного втручання людини.

**Безперервне розгортання: ТЕСТУВАННЯ – СТВОРЕННЯ
– РОЗГОРТАННЯ АВТОМАТИЧНО**

КОМБЕЕР CI/CD



КОНВЕЄР

Конвеєр — це група *завдань*, які виконуються *поетапно*.

Усі завдання на етапі виконуються паралельно (якщо є достатня кількість одночасно доступних *виконавців*)

Якщо всі завдання виконані успішно, конвеєр переходить до наступного етапу. Якщо одне із завдань не виконується, наступний етап (як правило) не виконується. Ви можете отримати доступ до сторінки конвеєрів на вкладці конвеєрів вашого проекту.

ЕТАПИ

stages використовується для визначення етапів, які можуть використовуватися завданнями, і визначається глобально.

Специфікація **ступенів** дозволяє мати гнучкі багатоступеневі конвеєри. Розташування елементів по етапах визначає порядок виконання робіт:

1. Завдання одного етапу виконуються паралельно.
2. Завдання наступного етапу запускаються після успішного завершення завдань попереднього етапу.

ЗАВДАННЯ

Файл YAML визначає набір завдань із обмеженнями, що вказують, коли їх слід запускати. Ви можете вказати необмежену кількість завдань, які визначаються як елементи верхнього рівня з довільним іменем і завжди мають містити принаймні пункт сценарію.

Завдання беруться виконавцями (Runners) і виконуються в середовищі виконавця. Важливо, що кожне завдання виконується незалежно одне від одного.

Кожне завдання має мати унікальну назву, але є кілька **зарезервованих ключових слів**, які не можна використовувати як назви завдань:

- image, services, stages, types, before_script, after_script, variables and cache

RUNNER

GitLab Runner — це проект із відкритим вихідним кодом, який використовується для запуску ваших завдань і надсилання результатів назад до GitLab.

Розроблено мовою Go.

Він використовується разом із GitLab CI, службою безперервної інтеграції з відкритим кодом, яка входить до GitLab і координує завдання.

ОСОБЛИВОСТІ RUNNER

- Дозволяє запускати:
 - кілька завдань одночасно
 - використовувати кілька токенів із кількома серверами (навіть для кожного проекту)
 - обмеження кількості одночасних завдань на маркер
- Завдання можна запускати:
 - локально
 - за допомогою контейнерів Docker
 - за допомогою контейнерів Docker і виконання завдання через SSH
 - за допомогою контейнерів Docker з автомасштабуванням на різних хмарах і гіпервізорах віртуалізації
 - підключившись до віддаленого SSH-сервера

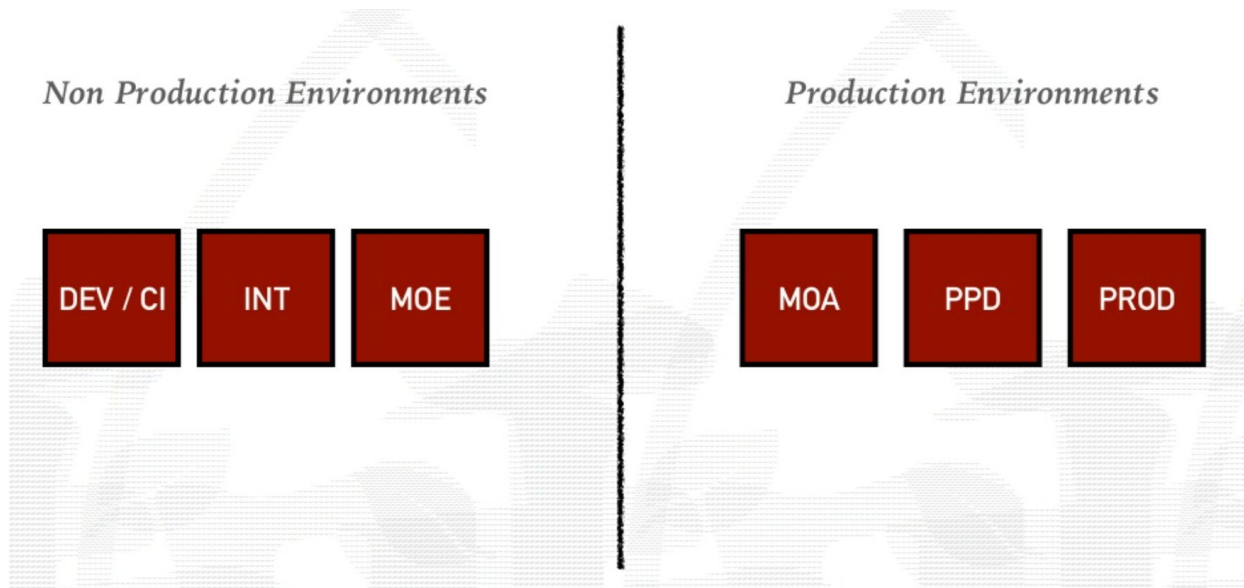
ОСОБЛИВОСТІ RUNNER

- Написано на Go та розповсюджується як єдиний двійковий файл без будь-яких інших вимог
- Підтримує Bash, Windows Batch і Windows PowerShell
- Працює в GNU/Linux, OS X і Windows (майже скрізь, де можна запустити Docker)
- Дозволяє налаштувати середовище виконання завдання
- Автоматичне перезавантаження конфігурації без перезапуску
- Просте у використанні налаштування з підтримкою робочих середовищ Docker, Docker-SSH, Parallels або SSH
- Вмикає кешування контейнерів Docker
- Легке встановлення як сервіс для GNU/Linux, OSX і Windows
- Вбудований HTTP-сервер метрик Prometheus

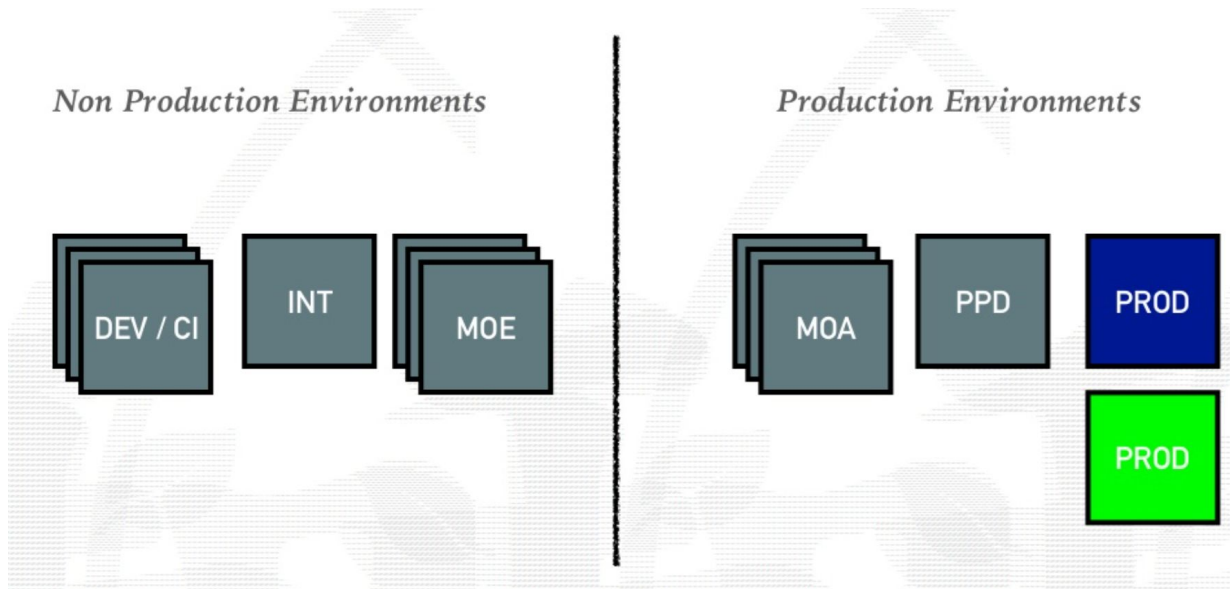


Конвеєри CI/CD

КЛАСИЧНА ІНФРАСТРУКТУРА



ΧΜΑΡΗΑ ΙΗΦΡΑΣΤΡΥΚΤΥΡΑ



Чому GitLab CI?

Інтеграція

Повністю інтегрований з GitLab

Легко почати

Кілька рядків у yml (YAML) усередині .gitlab-ci.yml і трохи клацань

Масштабований

Паралельні завдання (одночасне виконання), багато виконавців, позначені виконавці.

Ізольоване тестове середовище

Використання контейнерів Docker

Конфігурація GitLab CI

Виконується через файл `.gitlab-ci.yml`:

Приклад проекту NodeJS:

```
nodejs_run:  
  stage: test  
  script:  
    - npm install  
    - npm test
```



Використання GitLab CI

Спочатку отримайте виконавця

Простий Ubuntu Server VDS може виконувати цю роль.

Надайте його за допомогою сценарію:

```
# Gitlab CI multi runner
curl -L https://packages.gitlab.com/install/repositories/runner/gitlab-ci-multi-runner/
apt-get install -y gitlab-ci-multi-runner

echo 'run "gitlab-ci-multi-runner register"'
```

Запустіть **gitlab-ci-multi-runner register** і дайте відповіді на запитання. Ви можете знайти свій унікальний реєстраційний маркер у розділі **Налаштування** ---> **Виконавці**.

Додайте .gitlab-ci.yml до свого репо

Приклад для nodejs:

```
nodejs_run:  
  stage: test  
  script:  
    - npm install  
    - npm test
```



Професійні поради щодо використання

Використовуйте параметр кешу у вашому .gitlab-ci.yml, щоб зберегти каталоги або файли

Цей рядок буде кешувати всі невідстежувані файли git і файли в каталозі node_modules

```
cache:  
  untracked: true  
  paths:  
    - node_modules/
```

Пропуск збірки

Збірку буде пропущено, якщо ваше повідомлення коміту містить `[ci skip]`

Перевірте свій `.gitlab-ci.yml`

Використання літера: gitlab.com/ci/lint

Перегляньте стан свого конвеєра та завдань

Коли ви зафіксували свої зміни, почався конвеєр.

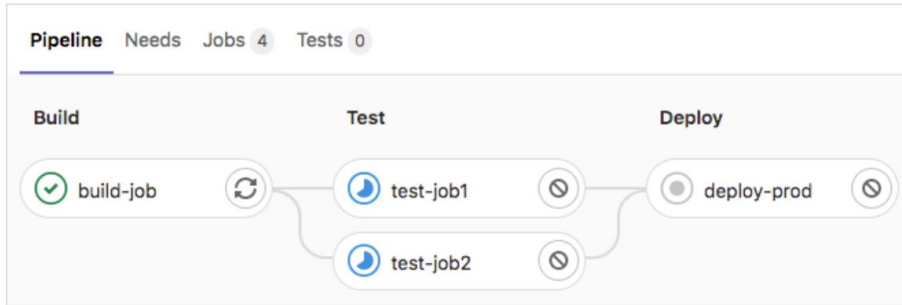
Щоб переглянути свій конвеєр:

- Go CI/CD > Pipelines.
Має бути відображено конвеєр з трьома етапами:

Status	Pipeline	Triggerer	Commit	Stages
 running	#217408060 latest		 master  271ad0cd  Update .gitlab-ci.yml	  

Перегляньте стан свого конвеєра та завдань

- Щоб переглянути візуальне представлення вашого конвеєра, клацніть його ідентифікатор.



Перегляньте стан свого конвеєра та завдань

- Щоб переглянути деталі завдання, клацніть назву завдання, наприклад, **deploy-prod**
- Якщо статус завдання застряг, перевірте, чи ймовірно налаштовано програму запуску для проекту.

✓ passed Job #855275091 triggered 23 minutes ago by  Suzanne Selhorn

```
1 Running with gitlab-runner 13.6.0-rc1 (d83ac56c)
2   on docker-auto-scale ed2dce3a
3   ✓ Preparing the "docker+machine" executor
4   Using Docker executor with image ruby:2.5 ...
5   Pulling docker image ruby:2.5 ...
6   Using docker image sha256:b7280b81558d31d64ac82aa66a9540e04baf9d15abb8fff
   ed62cd60e4fb5bf4132943d6fa2688 ...
7   ✓ Preparing environment
8   Running on runner-ed2dce3a-project-16381496-concurrent-0 via runner-ed2dce3a
9   ✓ Getting source from Git repository
10  $ eval "$CI_PRE_CLONE_SCRIPT"
11  Fetching changes with git depth set to 50...
12  Initialized empty Git repository in /builds/sselhorn/test-project/.git/
13  Created fresh repository.
14  Checking out 7353da73 as master...
15  Skipping Git submodules setup
16  ✓ Executing "step_script" stage of the job script
17  $ echo "This job deploys something from the $CI_COMMIT_BRANCH branch."
18  This job deploys something from the master branch.
19  ✓ Cleaning up file based variables
20  Job succeeded
```

Питання та відповіді